

User Manual

Bakery Shop

Internet Programming Project

Wipawin Tungwattanakittikul
6730202424



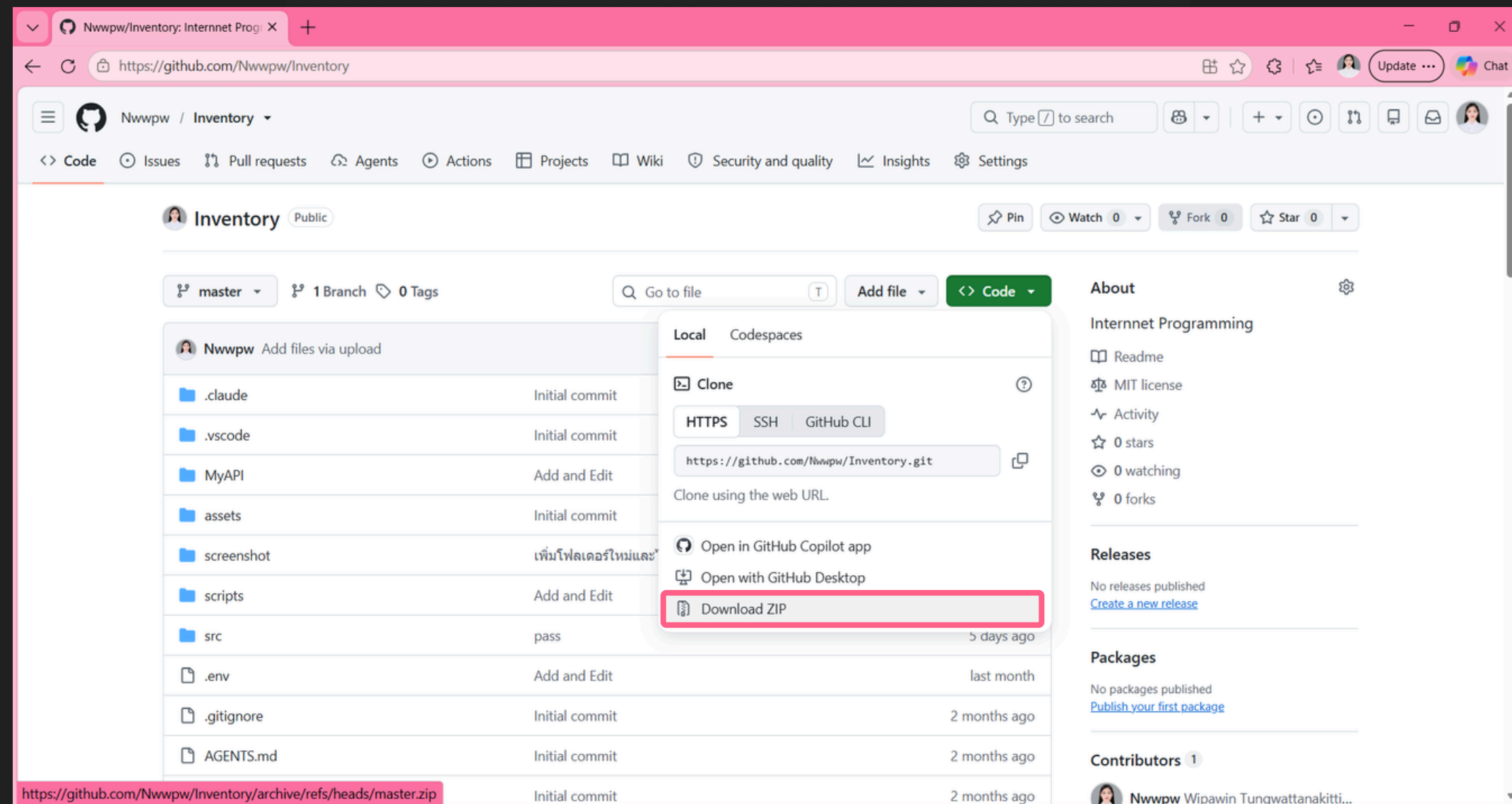
Chapter 1:

Installation & Setup

+

1.1 How to Download the Project from GitHub

1. Open the Inventory repository on GitHub.
2. Click the Code button.
3. Choose a download method:
 - Click Download ZIP
 - Extract the ZIP file after downloading
 - `git clone https://github.com/Nwwpw/Inventory.git`
4. Open the downloaded project folder in Visual Studio Code.



1.2 Required Software, Dependencies, and Configurations

Development & Execution Tools

Node.js & npm

Purpose:

Used to install project dependencies and run both frontend and backend components of the Bakery Inventory Management System.

Functions:

- Install required packages
- Run frontend and backend services
- Manage project dependencies

Visual Studio Code

Purpose:

Used as the primary development environment for coding, debugging, and project management.

Functions:

- Edit source code
- Manage project files
- Execute development commands

Expo

Purpose:

Used to develop, run, and test the React Native application.

Functions:

- Launch the application
- Test application features
- Preview the user interface

1.2 Required Software, Dependencies, and Configurations

Development & Execution Tools

PowerShell



Purpose

PowerShell is used to execute commands and manage development tasks during the frontend development process.

Functions

- Run project commands
- Install project dependencies
- Start the Expo development server
- Manage application files and folders

PuTTY



Purpose

PuTTY is used to establish a secure remote connection to the cloud server hosting the backend application.

Functions

- Access the cloud server remotely
- Execute backend commands
- Start and monitor the backend server
- Manage server-side application deployment

1.2 Required Software, Dependencies, and Configurations

Backend & Database

Express.js

Purpose:

Used to create RESTful APIs and handle communication between the frontend and database.

Functions:

- Process client requests
- Manage API endpoints
- Handle business logic

MySQL

Purpose:

Used to store and manage application data.

Functions:

- Store product information
- Store user accounts
- Store inventory records

phpMyAdmin

Purpose:

Used for database administration through a web interface.

Functions:

- Manage database tables
- Execute SQL queries
- Import and export database files

1.2 Required Software, Dependencies, and Configurations

Backend & Database

Git

Purpose:

Used for source code version control and project tracking.

Functions:

- Track source code changes
- Manage project versions
- Support development workflow

GitHub

Purpose:

Used to host and manage the project repository online.

Functions:

- Store source code
- Backup project files
- Share and manage project updates

1.2 Required Software, Dependencies, and Configurations

Backend & Database

React Native

Purpose

React Native is used to build the user interface of the Bakery Inventory Management System.

Functions

- Create application screens
- Manage user interactions
- Display inventory information
- Support cross-platform development

Expo

Purpose

Expo is used to develop, run, and test the React Native application.

Functions

- Launch the application
- Run the development server
- Test application features

Expo Router

Purpose

Expo Router is used to manage navigation and routing within the application.

Functions

- Navigate between screens
- Manage application routes
- Support page-based navigation

Expo

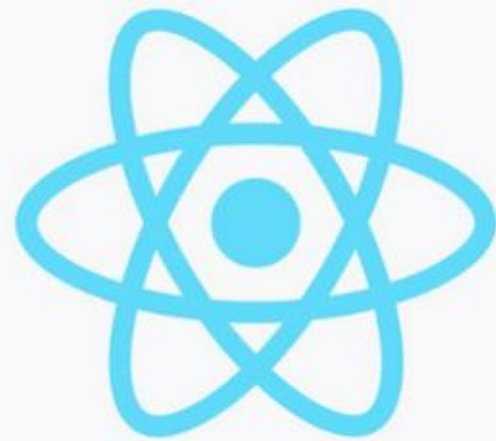
Purpose:

Used to develop, run, and test the React Native application.

Functions:

- Launch the application
- Test application features
- Preview the user interface

1.3 Frontend Dependencies



React Native

Purpose

React Native is used to build the user interface of the Bakery Inventory Management System.

Functions

- Create application screens
- Manage user interactions
- Display inventory information
- Support cross-platform development

```
PS C:\Users\user\Inventory> npx expo start
env: load .env
env: export DB_HOST DB_NAME DB_PASSWORD DB_F
Starting project at C:\Users\user\Inventory
> Port 8081 is being used by another process
✓ Use port 8082 instead? ... yes
Using src/app as the root directory for Expo
React Compiler enabled
Starting Metro Bundler
```

Expo

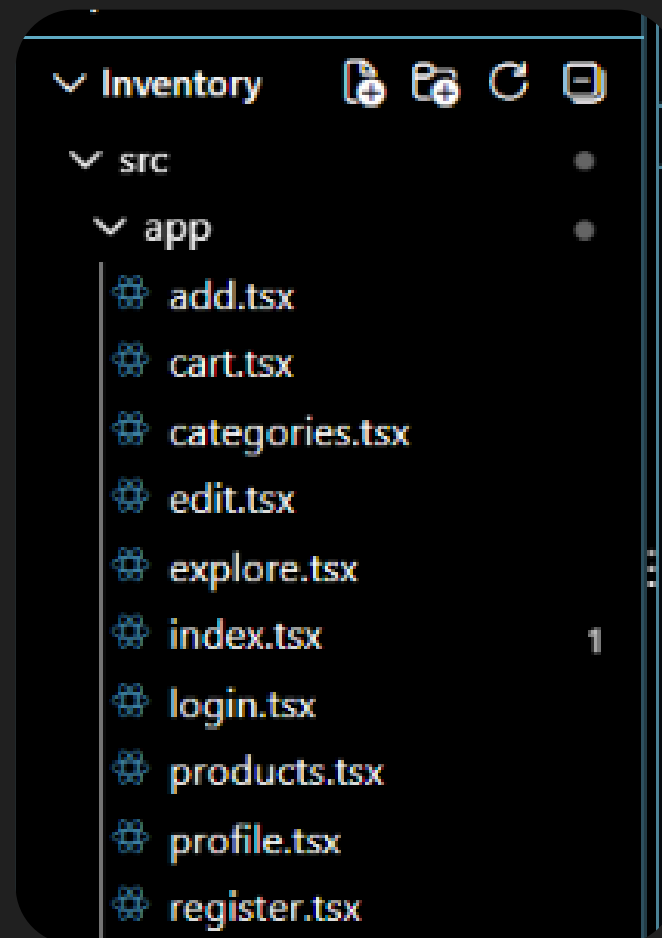
Purpose

Expo is used to develop, run, and test the React Native application.

Functions

- Launch the application
- Run the development server
- Test application features

1.3 Frontend Dependencies



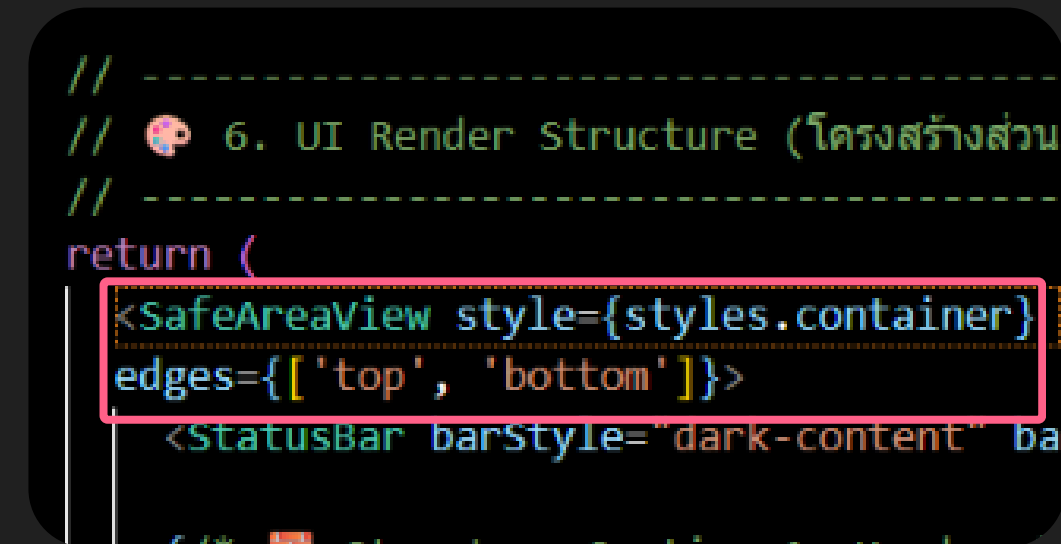
Expo Router

Purpose

Expo Router is used to manage navigation and routing within the application.

Functions

- Navigate between screens
- Manage application routes
- Support page-based navigation



React Native Safe Area Context

Purpose

Used to ensure content is displayed correctly within safe screen boundaries across different devices.

Functions

- Prevent content overlap
- Improve screen compatibility
- Support responsive layouts

1.4 Backend Core Technologies

Node.js

```
PuTTY (inactive)
login as: std6730202424
std6730202424@119.59.102.161's password:
Access denied
std6730202424@119.59.102.161's password:
Access denied
std6730202424@119.59.102.161's password:
IP course shell: std6730202424
Workspace: /app
Assigned backend port: 3041
Node.js: npm ci then node server.js
FastAPI: fastapi-init then ./run.sh
FastAPI docs: http://119.59.102.161:3041/docs
std6730202424@ip-course:~$ node server.js
Public backend URL (after successful startup): http://119.59.102.161:
◊ injected env (6) from .env // tip: □ suppress logs { quiet: true }
🚀 API running on port 3041
Connected to MySQL: ip_std6730202424
USER DATA = {
  id: 1,
```

Purpose

Node.js is used as the runtime environment for executing server-side JavaScript code.

Functions

- *Run backend services*
- *Process client requests*
- *Communicate with the database*
- *Handle API operations*

1.4 Backend Core Technologies

Express.js

```
app.put('/api/products/:id', authenticateToken,  
  });
```

```
// ★ 4. DELETE PRODUCT (DELETE)
```

```
app.delete('/api/products/:id',  
  authenticateToken, requireAdmin, async (req,  
    const { id } = req.params;
```

```
  try {  
    if (!dbConnected) {  
      let products = getProductsFromJSON();  
      products = products.filter(p => p.id !== i
```

Purpose

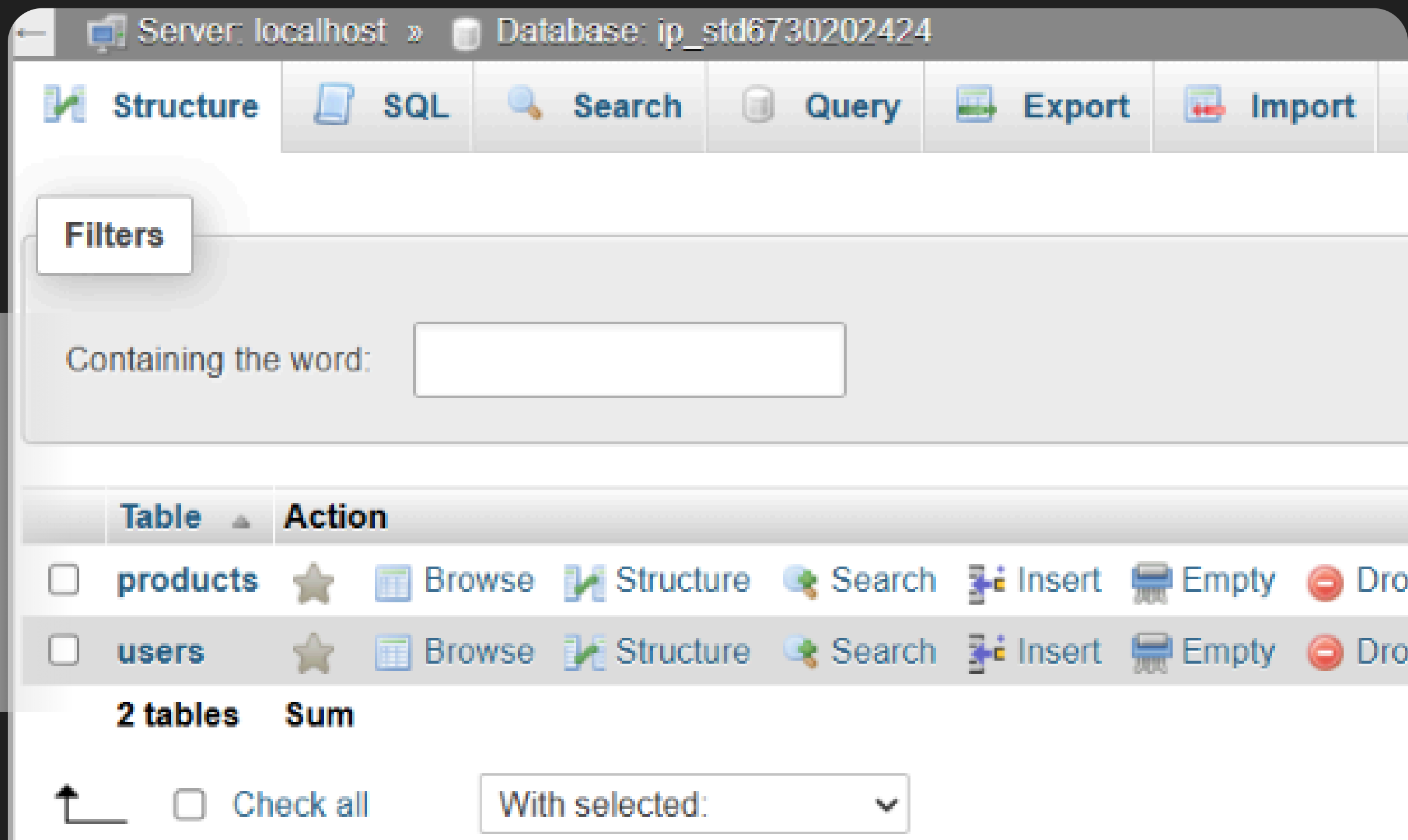
Express.js is used to create RESTful APIs and manage server-side routing.

Functions

- Create API endpoints
- Process HTTP requests and responses
- Handle business logic
- Connect the frontend with the database

1.4 Backend Core Technologies

MySQL



Purpose

MySQL is used to store and manage application data.

Functions

- *Store user information*
- *Store product information*
- *Store inventory records*
- *Manage application data*

1.4 Backend Core Technologies

JWT Authentication

```
jwt.verify(token, jwtSecret, (err, user) => {
  if (err) {
    return res.status(403).json({
      error: 'Invalid token'
    });
  }
});
```

```
const token = jwt.sign(
  {
    id: user.id,
    username: user.username,
    role: user.role
  },
  jwtSecret,
  {
    expiresIn: '1h'
  }
);
```

Purpose

JWT Authentication is used to verify user identity and secure API access.

Functions

- Generate authentication tokens
- Verify user sessions
- Protect API endpoints
- Control user access permissions

1.4 Backend Core Technologies

CORS

```
const port = process.env.PORT
```

```
app.use(cors({ origin: '*' } )
```

```
app.use(express.json({ limit
```

```
const dbConfig = {
```

```
  host: process.env.DB_HOST,
```

Purpose

CORS is used to allow communication between the frontend and backend applications.

Functions

- Enable cross-origin requests
- Support frontend-backend communication
- Improve API accessibility

1.4 Backend Core Technologies

dotenv

⚙️ .env

```
1  PORT=3041
2  DB_HOST=localhost
3  DB_USER=std6730202424
4  DB_PASSWORD=*****
5  DB_NAME=ip_std6730202424
6  DB_PORT=3306
7  JWT_SECRET=mysecretkey
8
```

Purpose

dotenv is used to manage environment variables and sensitive application configurations.

Functions

- Store database credentials
- Store server configuration
- Store JWT secret keys
- Improve application security

1.5 Backend Configuration

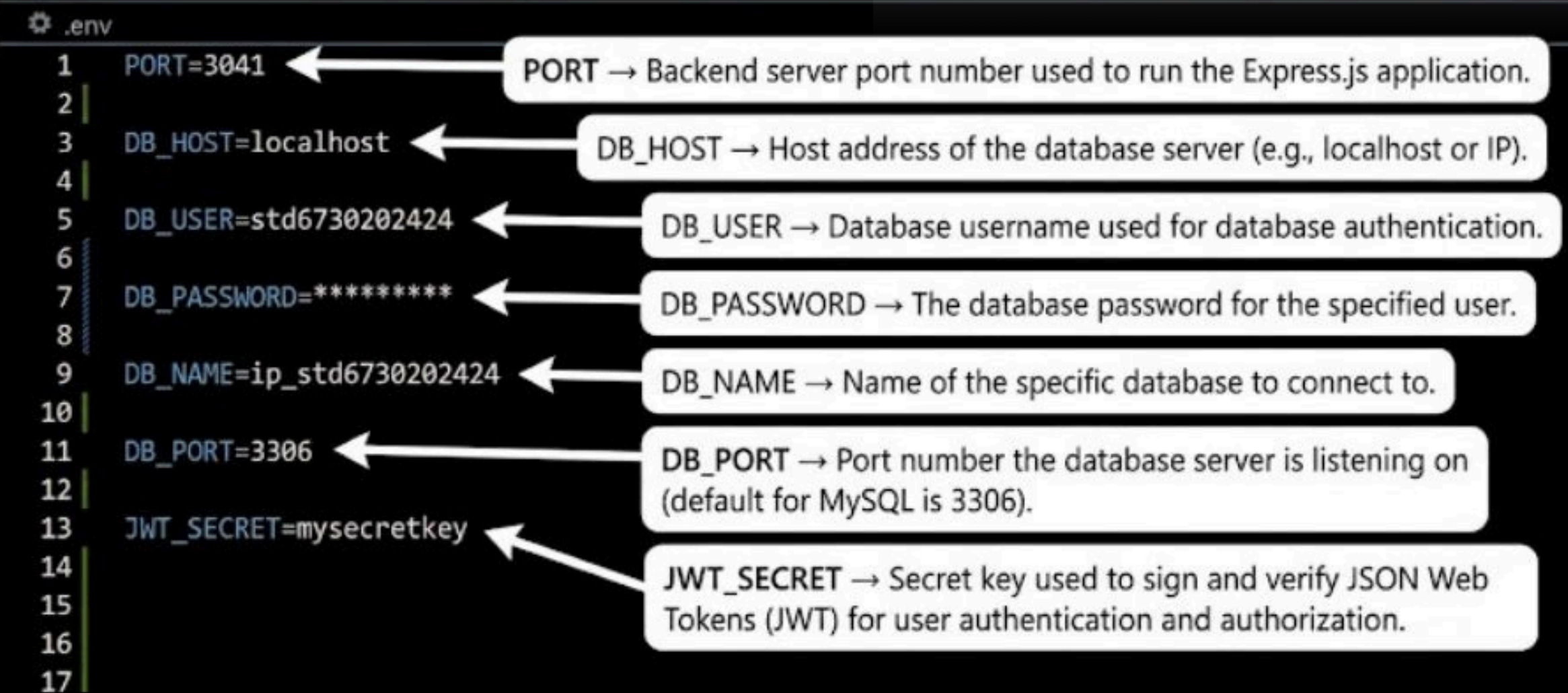
Environment Variables

Purpose

Environment variables are used to store sensitive application settings and backend configurations.

Functions

- Store database connection information
- Configure backend server settings
- Manage application security settings
- Store JWT authentication secrets



The diagram shows a code editor window with a file named `.env`. The file contains seven lines of configuration. Each line is annotated with a callout box explaining the variable's purpose. Arrows point from the callout boxes to the corresponding lines in the code.

```
1 PORT=3041
2
3 DB_HOST=localhost
4
5 DB_USER=std6730202424
6
7 DB_PASSWORD=*****
8
9 DB_NAME=ip_std6730202424
10
11 DB_PORT=3306
12
13 JWT_SECRET=mysecretkey
14
15
16
17
```

PORT → Backend server port number used to run the Express.js application.

DB_HOST → Host address of the database server (e.g., localhost or IP).

DB_USER → Database username used for database authentication.

DB_PASSWORD → The database password for the specified user.

DB_NAME → Name of the specific database to connect to.

DB_PORT → Port number the database server is listening on (default for MySQL is 3306).

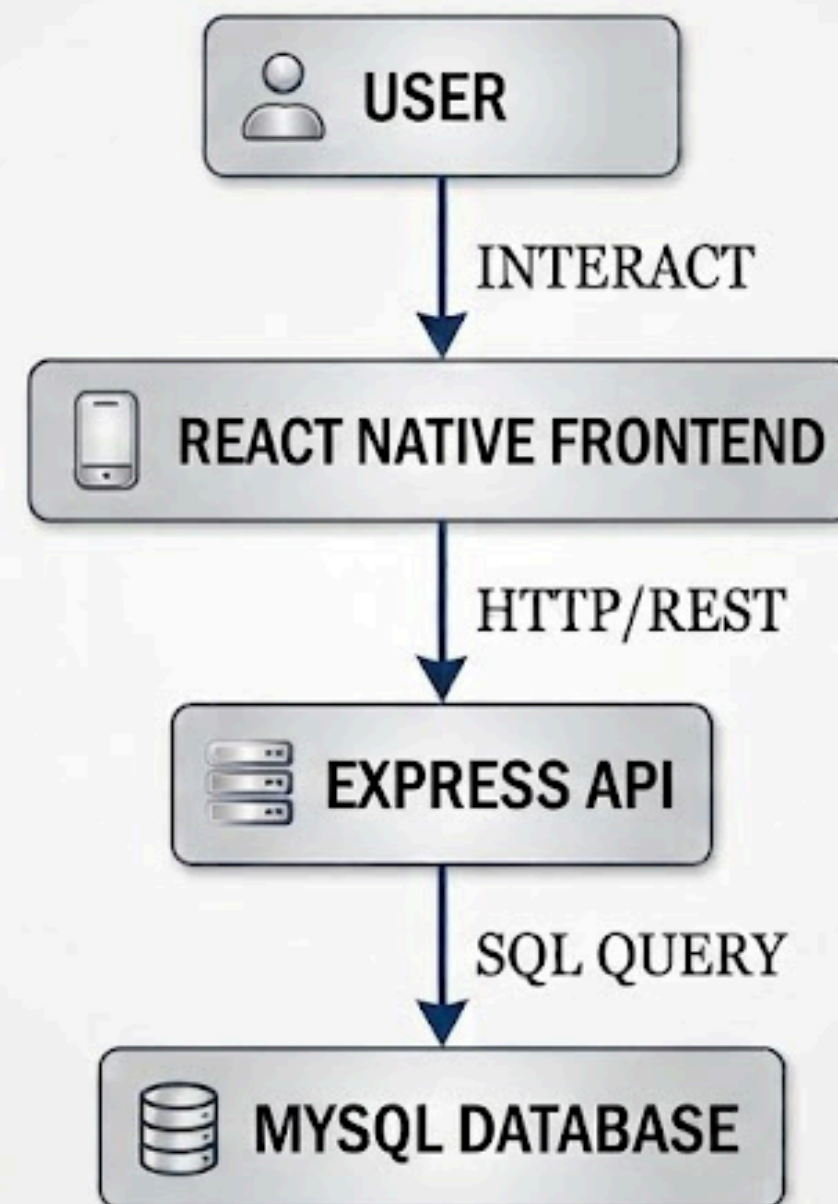
JWT_SECRET → Secret key used to sign and verify JSON Web Tokens (JWT) for user authentication and authorization.

1.5 Backend Configuration

System Architecture

The Inventory Management System follows a client-server architecture. Users interact with the system through the React Native frontend application. User requests are processed by the Express.js backend through REST APIs, while the MySQL database stores and manages user accounts, product information, and inventory records. The processed data is then returned to the frontend and displayed to the user.

SYSTEM ARCHITECTURE



1.6 Backend Project Structure

Main Files

server.js

Purpose

The main backend application file responsible for handling REST API requests, user authentication, database communication, and inventory management operations.

Functions

- Create REST API endpoints
- Process client requests
- Handle JWT authentication
- Communicate with the MySQL database
- Manage product and user data

JS server.js > 🔍 authenticateToken

```
1  require('dotenv').config();
2  const express = require('express');
3  const cors = require('cors');
4  const mysql = require('mysql2/promise');
5  const jwt = require('jsonwebtoken');
6  const fs = require('fs');
7  const path = require('path');
8
9  const app = express();
10 const port = process.env.PORT || 3041;
11
12 app.use(cors({ origin: '*' }));
13 app.use(express.json({ limit: '5mb' }));
14
15 const dbConfig = {
```

```

{} products.json > {} 11
1  [
2  {
3    "id": 1786125491456,
4    "name": "Lemon Meringue Pie",
5    "category": "Pies & Tarts",
6    "price": 65,
7    "stock": 5,
8    "image": "https://www.glutenfreepalate.com/wp-content/uploa
9  },
10 {
11  "id": 1786121075152,
12  "name": "Caramel Pie",
13  "category": "Pies & Tarts",
14  "price": 65,
15  "stock": 15,
16  "image": "https://tse3.mm.bing.net/th/id/OIP.YbNxbBnetDX75M
17  },
18  {

```

1.6 Backend Project Structure

Main Files

products.json

Purpose

Used as a backup data source when the MySQL database is unavailable.

Functions

- Store product information in JSON format
- Provide fallback data storage
- Support inventory data retrieval

1.6 Backend Project Structure

Main Files

package.json

Purpose

Contains project metadata, dependencies, and scripts required to run the backend application.

Functions

- Manage backend dependencies
- Define project configuration
- Provide execution scripts

```
"dependencies": {  
  "@expo/ui": "~57.0.4",  
  "@expo/vector-icons": "^15.0.2",  
  "@react-native-async-storage/async-storage": "2.2.0",  
  "@react-navigation/native": "^7.3.15",  
  "bcrypt": "^6.0.0",  
  "cors": "^2.8.6",  
  "dotenv": "^17.4.2",  
  "expo": "~57.0.4",  
  "expo-constants": "~57.0.3",  
  "expo-device": "~57.0.0",  
  "expo-font": "~57.0.0",  
  "expo-glass-effect": "~57.0.0",  
  "expo-image": "~57.0.0",  
  "expo-image-picker": "~57.0.13",  
  "expo-linking": "~57.0.2".  
}
```

1.6 Backend Project Structure

Main Files

.env

Purpose

Stores environment variables and sensitive configuration settings required by the backend application.

Functions

- Store database credentials
- Store server configuration
- Store JWT authentication settings
- Improve application security

```
gear .env
1  PORT=3041
2  DB_HOST=localhost
3  DB_USER=std6730202424
4  DB_PASSWORD=*****
5  DB_NAME=ip_std6730202424
6  DB_PORT=3306
7  JWT_SECRET=mysecretkey
8
9
```

1.7 Database Setup

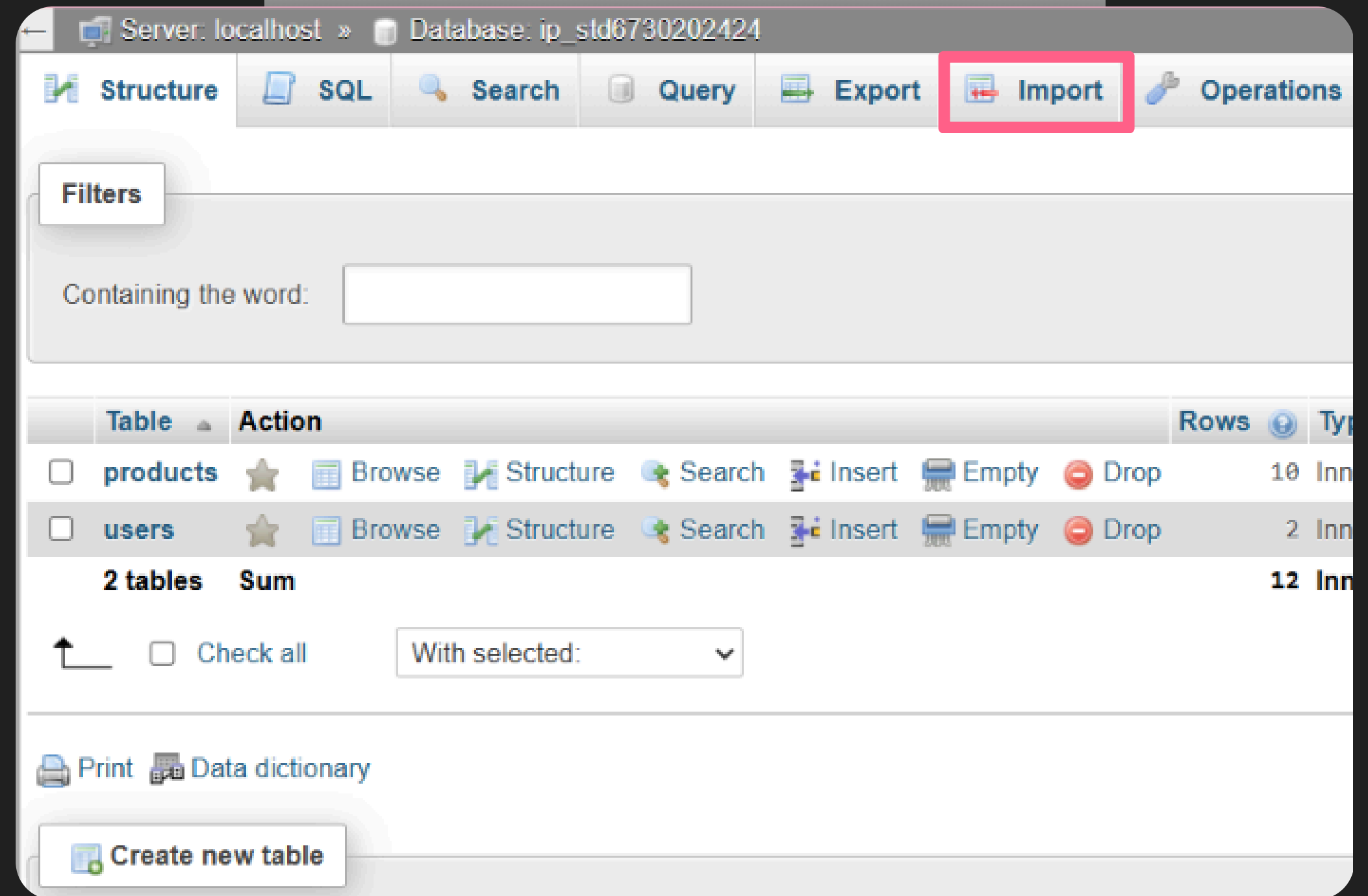
Import SQL File

Purpose

The SQL file is imported into MySQL to create the database structure and prepare the system for storing application data.

Steps

1. Open phpMyAdmin.
2. Create or select the database.
3. Click the Import tab.
4. Choose the SQL file.
5. Click Import to start the database import process.
6. Wait until the import is completed successfully.



1.7 Database Setup

Verify Tables

Purpose

Database verification ensures that all required tables have been created successfully after importing the SQL file.

Required Tables

USERS

Stores user account information and authentication data.

PRODUCTS

Stores product and inventory information used by the application.

	#	Name	T
☐	1	id 	in
☐	2	username 	v
☐	3	email	v
☐	4	password	v
☐	5	role	v
☐	6	profile_image	lo
☐	7	created_at	ti

	#	Name	T
☐	1	Id 	in
☐	2	name	v
☐	3	category	v
☐	4	stock	in
☐	5	price	in
☐	6	image	v



1.8 Run Backend

Purpose

The backend server is responsible for processing API requests, handling user authentication, communicating with the MySQL database, and managing inventory data.

Steps

1. Open PuTTY and connect to the cloud server.
2. Navigate to the backend project directory.
3. Execute the following command: `node server.js`
4. Verify that the backend server starts successfully.

Expected Output:

- Connected to MySQL
- API running on port 3041

Functions

- Start the Express.js backend server
- Enable REST API services
- Connect to the MySQL database
- Process user and product requests
- Support frontend-backend communication

```
PuTTY (inactive)
login as: std6730202424
std6730202424@119.59.102.161's password:
Access denied
std6730202424@119.59.102.161's password:
Access denied
std6730202424@119.59.102.161's password:
IP course shell: std6730202424
Workspace: /app
Assigned backend port: 3041
Node.js: npm ci then node server.js
FastAPI: fastapi-init then ./run.sh
FastAPI docs: http://119.59.102.161:3041/d
std6730202424@ip-course:~$ node server.js
Public backend URL (after successful start
◊ injected env (6) from .env // tip: □ sup
🚀 API running on port 3041
Connected to MySQL: ip_std6730202424
```

1.9 Run Frontend

Purpose

The frontend application is executed using Expo to provide the user interface and allow users to access the Inventory Management System.

Steps

1. Open the project folder in Visual Studio Code.
2. Open the integrated terminal.
3. Install the required dependencies.
`npm install`
4. Start the Expo development server.
`npx expo start`
5. Wait for the Metro Bundler to launch successfully.
6. Open the application using a web browser or a mobile device through Expo.

Expected Output:

After the Expo development server starts successfully, the Metro Bundler will be displayed and the application will be available for testing and development.

Functions

- Start the frontend application
- Launch the Expo development server
- Build and bundle application resources
- Enable application testing and debugging

```
> inventory@1.0.0 web
> expo start --web
```

```
env: load .env
env: export DB_HOST DB_NAME DB_PASSWORD DB_PORT DB_USER JWT
Starting project at C:\Users\user\Inventory
Using src/app as the root directory for Expo Router.
React Compiler enabled
Starting Metro Bundler
```



```
> Scan the QR code above to open in Expo Go.
> Metro: exp://10.75.38.98:8081
> Web: http://localhost:8081
```

```
> Using Expo Go (Press s to switch to development build)
> Press ? | show all commands
```

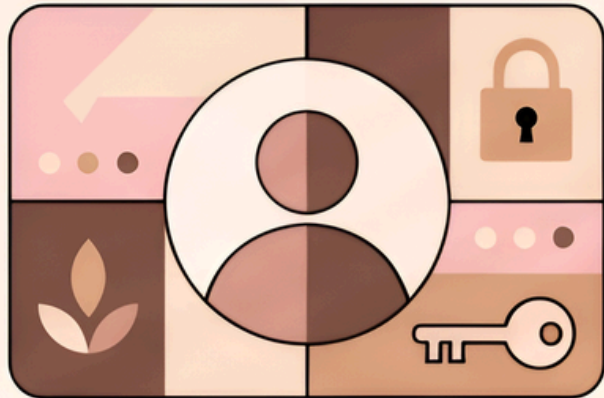
```
An update for expo is available: 57.0.4 → ~57.0.20
18 other packages may need updating. Run npx expo install -
Logs for your project will appear below. Press Ctrl+C to ex
λ Bundled 4155ms node_modules\expo\node_modules\@expo\cli\n
"shadow*" style props are deprecated. Use "boxShadow".
Web Bundled 5756ms node_modules\expo-router\entry.js (1257
Web Bundled 1369ms node_modules\expo-router\entry.js (1261
Web INFO %cDownload the React DevTools for a better devel
```

Chapter 2:

Application

Navigation Flow

Inventory Management System: Role-Based Access Control



JWT Authentication

Users access the system using a username and password secured by JWT Authentication.

STAFF USER PERMISSIONS



VIEW & SEARCH PRODUCTS



MANAGE PERSONAL PROFILE

ADMINISTRATIVE OVERSIGHT



MANAGE PRODUCTS



CONTROL INVENTORY DATA



MAINTAIN ALL SYSTEM OPERATIONS

PERMISSION MATRIX

FEATURE	STAFF	ADMIN
VIEW & SEARCH PRODUCTS	✓	✓
UPDATE PERSONAL PROFILE	✓	✓
ADD/EDIT/DELETE PRODUCTS	✗	✓
MANAGE INVENTORY	✗	✓

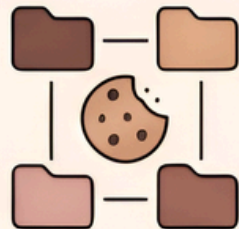


RESTRICTED MANAGEMENT FUNCTIONS

Product management and inventory controls are strictly restricted to administrative roles.

GRANULAR ROLE DEFINITION

Permissions are assigned to roles rather than individuals to simplify auditing.



Chapter 3:

User Pages

3.1 Login Page

The Login Page allows users to securely access the Inventory Management System using their username and password.

Features & Functions:

- Username Field
- Password Field
- Login Button
- Authentication Validation

How to Use:

Enter Username



Enter Password



Click Login



Access Home Page



3.2 Register Page

The Register Page allows new users to create an account for accessing the Inventory Management System.

Features & Functions:

- Profile Image Upload (Upload a profile picture during registration.)
- Automatic Profile Avatar (If no profile image is uploaded, the system automatically generates a placeholder avatar using the first character of the username.)
- Username (Enter a unique username for the account.)
- Email (Enter a valid email address.)
- Password (Create a secure password for the account.)
- Register Button (Create a new account and store user information in the database.)

How to Use:

Enter User Information



Create Account



Login to System

 **Register**



 Upload Photo

Username

Enter username

Email

example@email.com

Password

Enter Password

Confirm password

Confirm password

Sign up

Already have an account? [Log in](#)

3.3 Home Page

The Home Page displays product inventory information and provides access to search and navigation features.

Features & Functions:

- View Product List
- Search Products
- Language Switching
- Product Information Display
- Inventory Status Display

How to Use:

Login



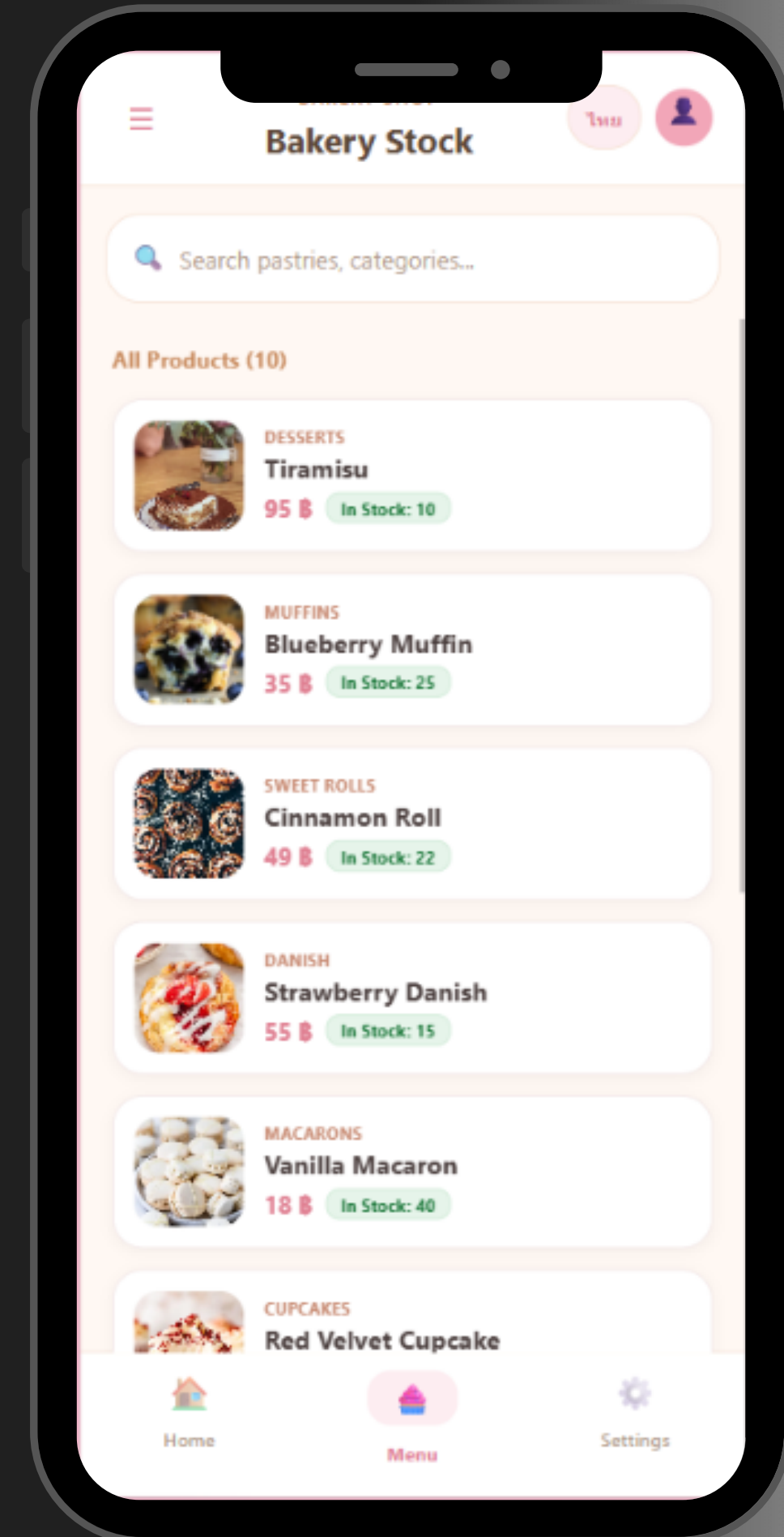
Open Home Page



Browse Products



Search Products



3.4 Profile Page

The Profile Page allows users to view and update their personal account information.

Features & Functions:

- Show Profile Image
- Show Role
- Show Username
- Show Email
- Password Verification
- Change Personal Information
- Save Changes
- Logout

The Profile Page allows users to view and manage their account information securely.

How to Use:

Open Profile



Click Edit Information



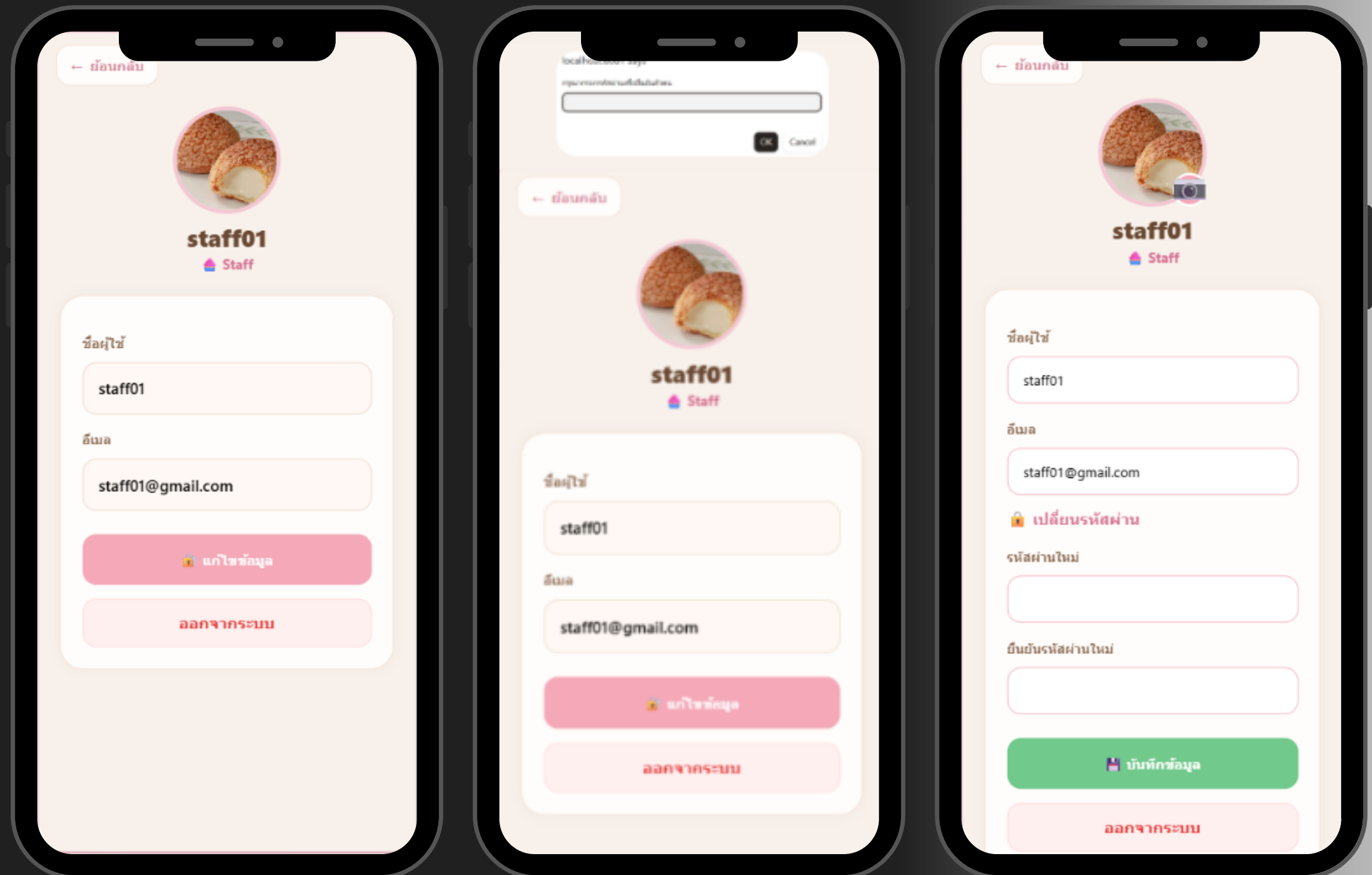
Verify Current Password



Edit Personal Information



Save Changes



Chapter 4:

**Administrative
Functions**

4.1 Product Management Page

The Product Management Page allows administrators to view, search, and manage all products within the Inventory Management System.

Features & Functions:

- View Product Information
- Search Products
- Edit Product Information
- Delete Products
- Monitor Inventory Status

How to Use:

Open Product Management Page



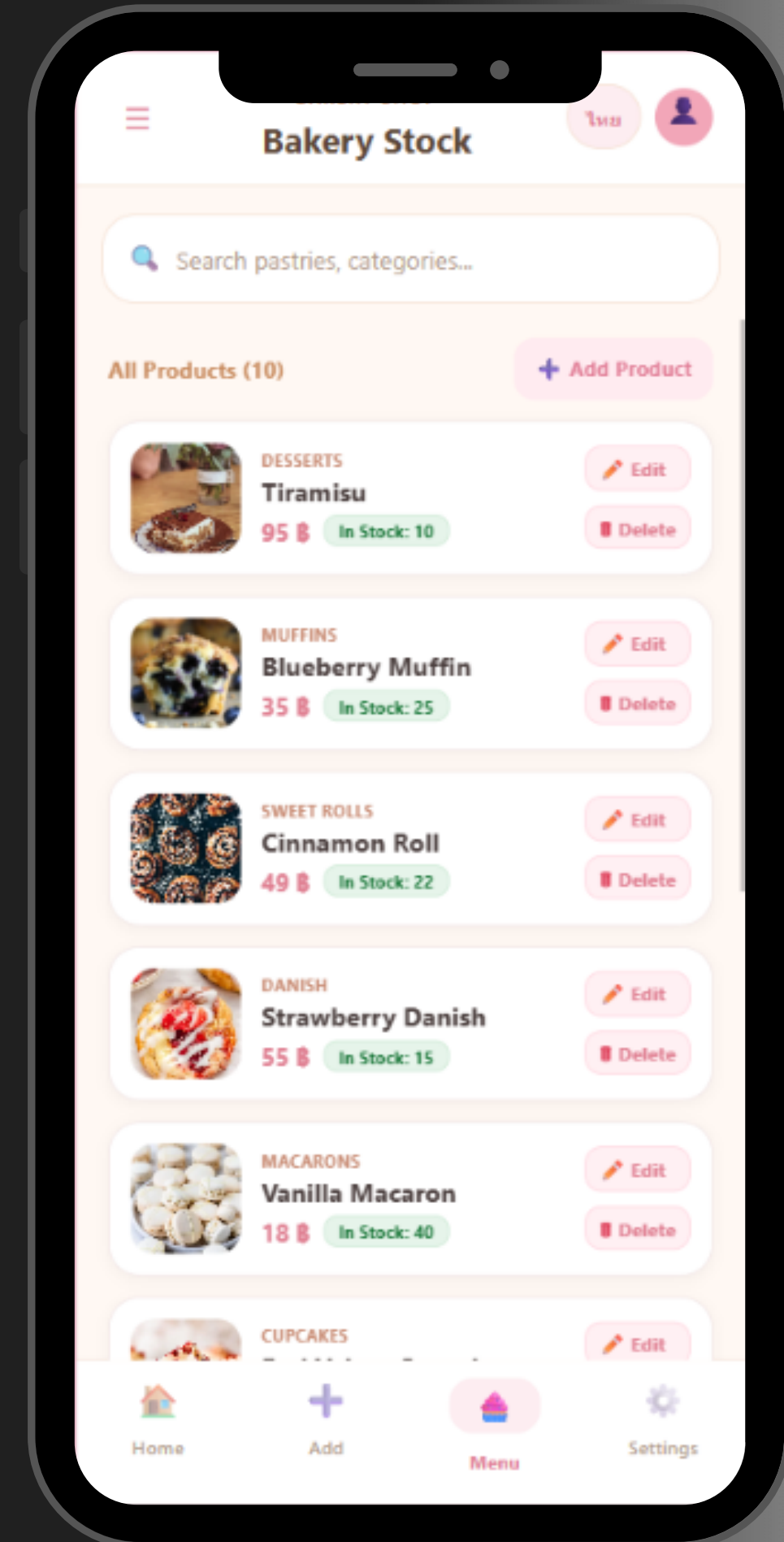
Search or View Products



Select Product



Edit or Delete Product



4.2 Add Product Page

The Add Product Page allows administrators to add new products to the inventory database.

Features & Functions:

- Product Name
- Category
- Price
- Stock Quantity
- Product Image Upload
- Create Product

How to Use:

Open Add Product Page



Enter Product Information



Upload Product Image



Click Save



Product Added Successfully

← Back Add New Pastry

Categories

Brownies

Or select one or more existing categories:

Cakes Cookies Croissants **Brownies**

Price (THB) *

39

Stock Quantity (pcs) *

7

Product Image

<https://i.pinimg.com/originals/02/c6/>

Preview:

+ Save New Product

☰ Bakery Stock You

Search pastries, categories...

All Products (11) + Add Product

- BROWNIES**
Cheesecake red velvet s...
39 ฿ In Stock: 7 Edit Delete
- DESSERTS**
Tiramisu
79 ฿ In Stock: 10 Edit Delete
- MUFFINS**
Blueberry Muffin
35 ฿ In Stock: 25 Edit Delete
- SWEET ROLLS**
Cinnamon Roll
49 ฿ In Stock: 22 Edit Delete
- DANISH**
Strawberry Danish
55 ฿ In Stock: 15 Edit Delete
- MACARONS**
... Edit

Home Add **Menu** Settings

4.3 Edit Product Page

The Edit Product Page allows administrators to update existing product information and inventory records.

Features & Functions:

- Update Product Name
- Update Category
- Update Price
- Update Stock Quantity
- Update Product Image
- Save Changes

How to Use:

Select Product



Open Edit Page



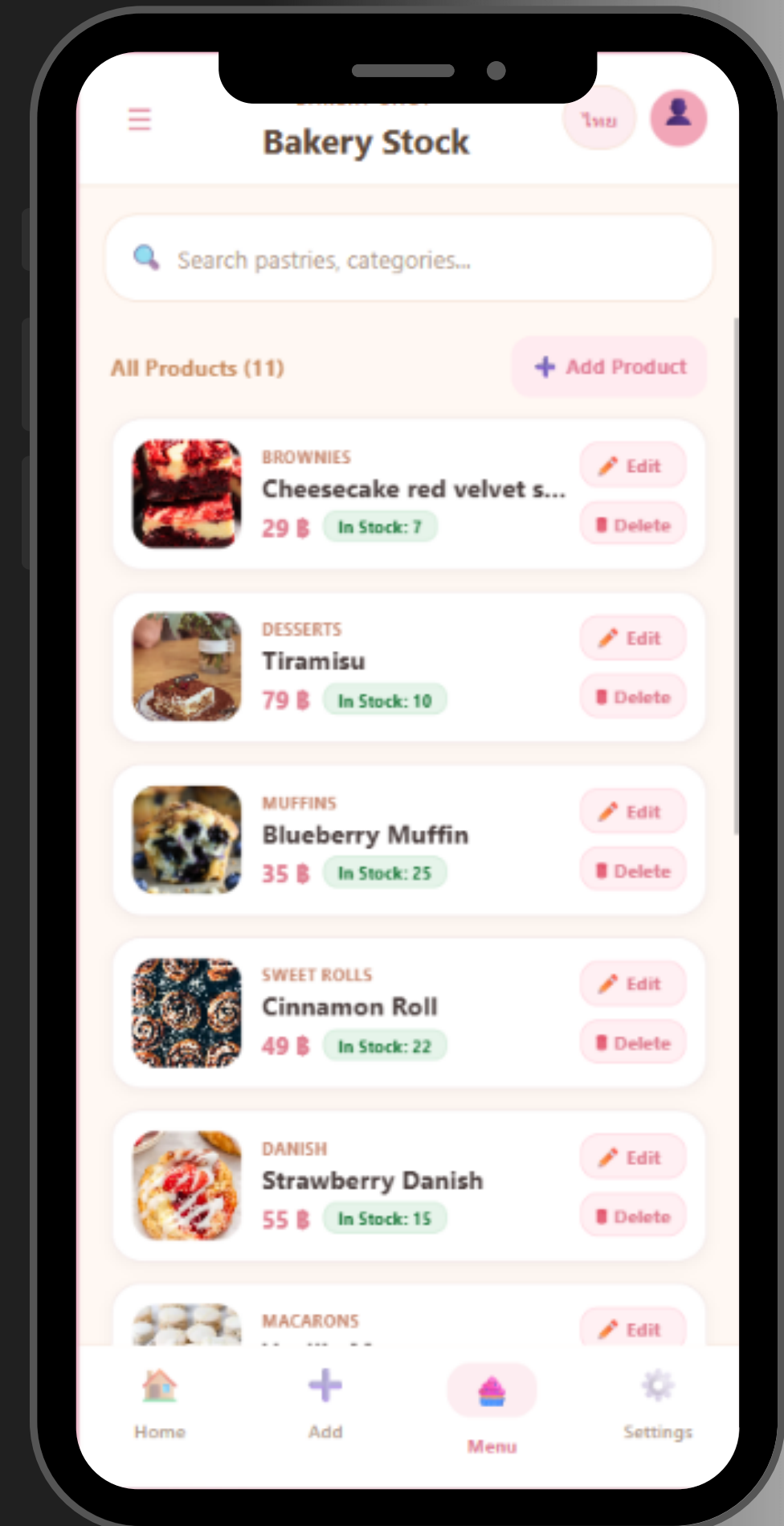
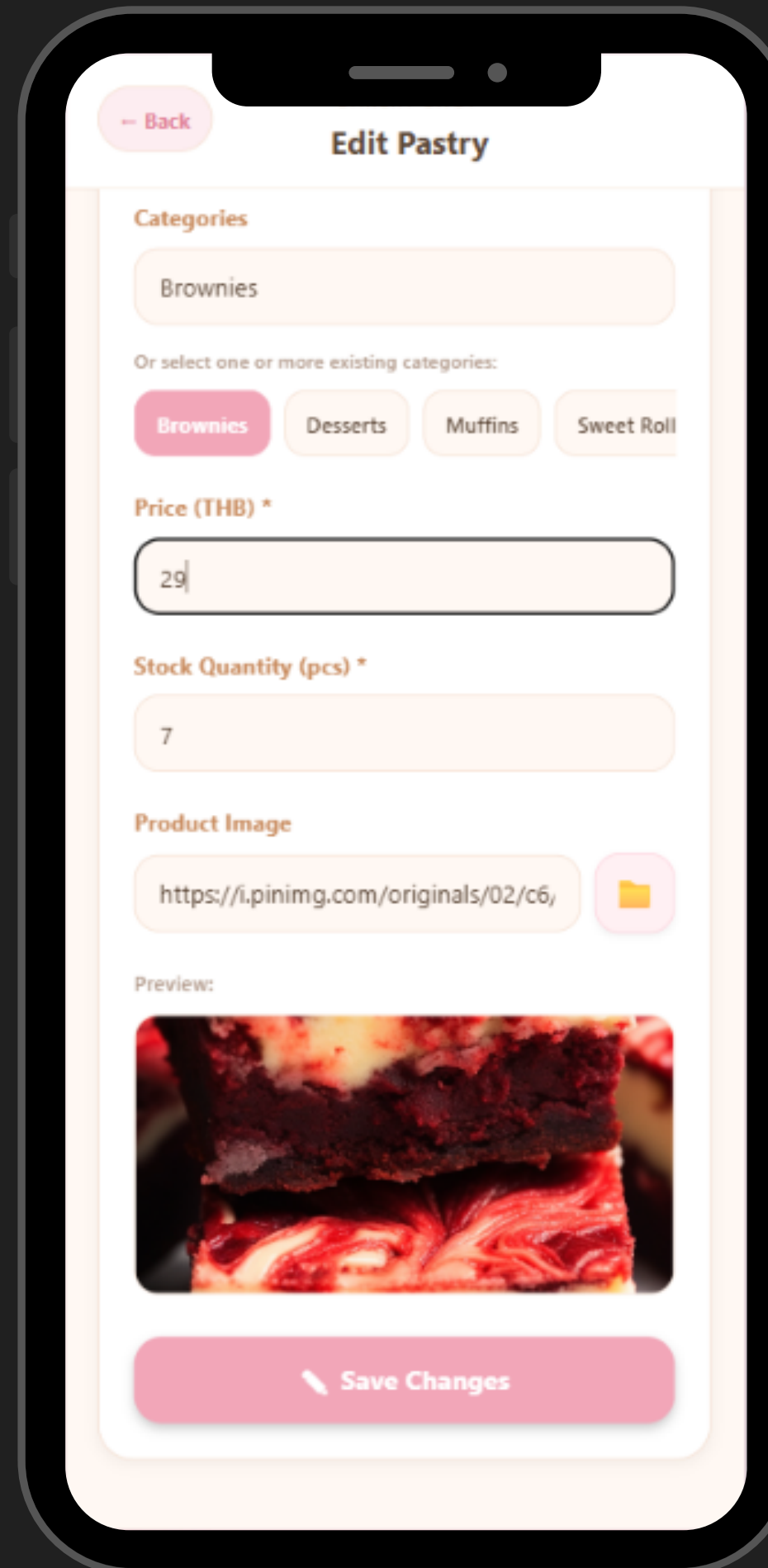
Modify Product Information



Save Changes



Product Updated Successfully



Chapter 5:

**Additional
Features**

5.1 Language Switching

The Inventory Management System supports bilingual user interfaces, allowing users to switch between English and Thai languages.

Available Languages

- English
- Thai

Features & Functions:

- Improve accessibility for users
- Support multilingual operation
- Enhance user experience

How to Use:

Open Home Page



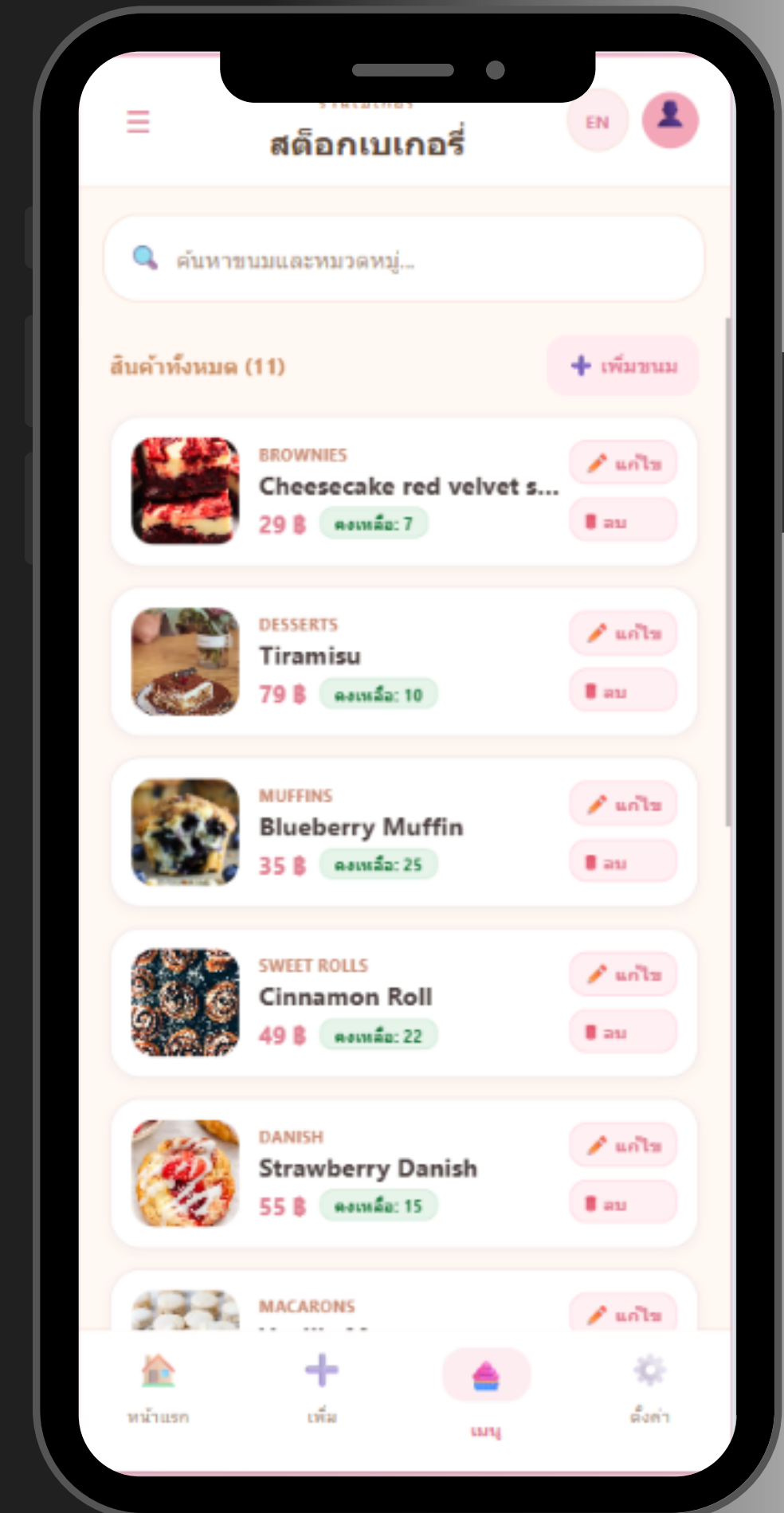
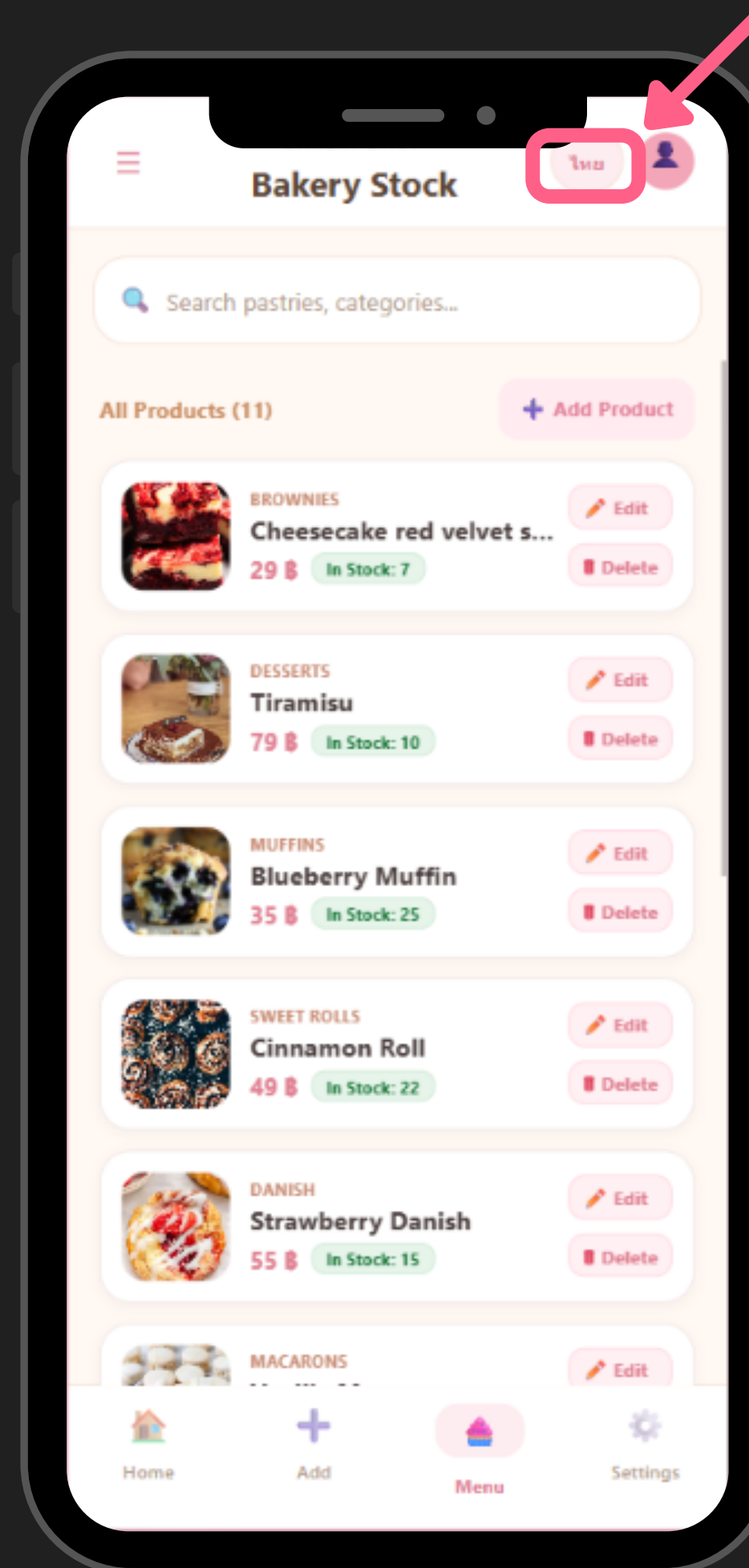
Click Language Switch Button



Select English or Thai



System Language Updated



5.2 Automatic Profile Avatar

The system automatically generates a profile avatar using the first character of the username when no profile image is uploaded.

Features & Functions:

- Automatic Avatar Generation
- Username-Based Placeholder
- Improved User Experience

How to Use:

Register Account



No Profile Image Uploaded



Extract First Character of Username



Generate Automatic Avatar



Display Avatar Based on the First Character of the Username

 **Register**





Username
Staff01

Email
example@email.com

Password
Enter Password

Confirm password
Confirm password

Sign up

[Already have an account? Log in](#)

"S" is generated automatically

Username

 Staff01

5.3 Password Verification Before Editing

Users must verify their current password before modifying personal account information.

Features & Functions:

- Identity Verification
- Profile Protection
- Secure Account Modification

How to Use:

Open Profile



Click Edit



Enter Current Password



Verify Identity



Edit Information

localhost:8081 says

กรุณากำหนดรหัสผ่านเพื่อยืนยันตัวตน

OK Cancel

Before Verification

← ย้อนกลับ

staff01
Staff

ชื่อผู้ใช้
staff01

อีเมล
staff01@gmail.com

แก้ไขรหัสผ่าน

ออกจากระบบ

After Verification

← ย้อนกลับ

staff01
Staff

ชื่อผู้ใช้
staff01

อีเมล
staff01@gmail.com

เปลี่ยนรหัสผ่าน

รหัสผ่านใหม่

ยืนยันรหัสผ่านใหม่

บันทึกข้อมูล

ออกจากระบบ

5.4 Category Memory System

The system automatically saves previously created product categories and allows administrators to reuse existing categories when adding new products.

Features & Functions:

- Category Reuse
- Automatic Category Storage
- Faster Product Creation
- Reduced Data Entry Errors

How to Use:

Create Product



Save New Category



Category Stored



Display Category Suggestions



Reuse Existing Category

Mobile app interface for "Edit Pastry". The form includes fields for Product Name (Tiramisu), Categories (Desserts), Price (THB) (79), Stock Quantity (pcs) (10), and Product Image (https://images.unsplash.com/photo-...). A red box highlights the "Categories" section, which includes a text input with "Desserts" and a list of existing categories: Brownies, Desserts, Muffins, and Sweet Roll. A red arrow points from this section to a separate "Previously Saved Categories" panel on the right.

Categories

Desserts

Or select one or more existing categories:

Brownies Desserts Muffins Sweet Roll

Previously Saved Categories

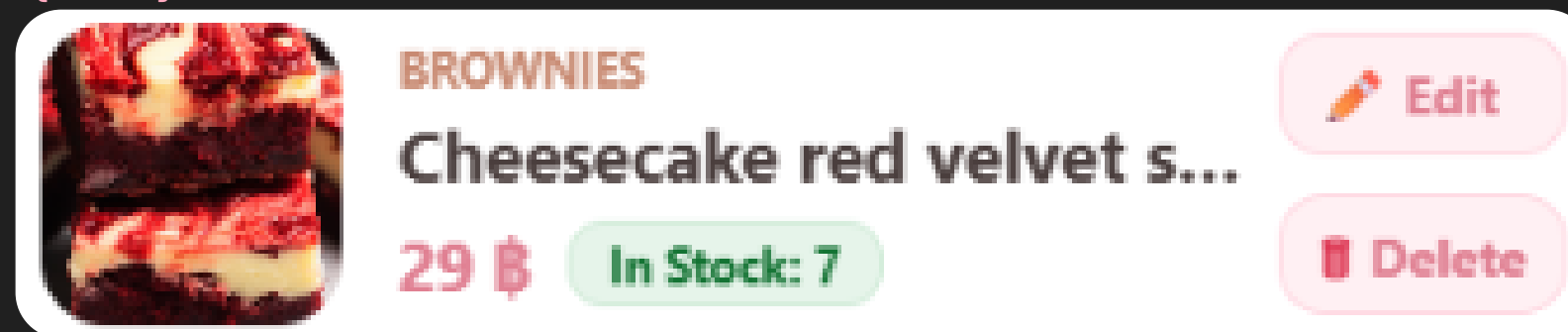
5.5 Inventory Stock Tracking

The system provides real-time inventory tracking by displaying current stock quantities for each product.

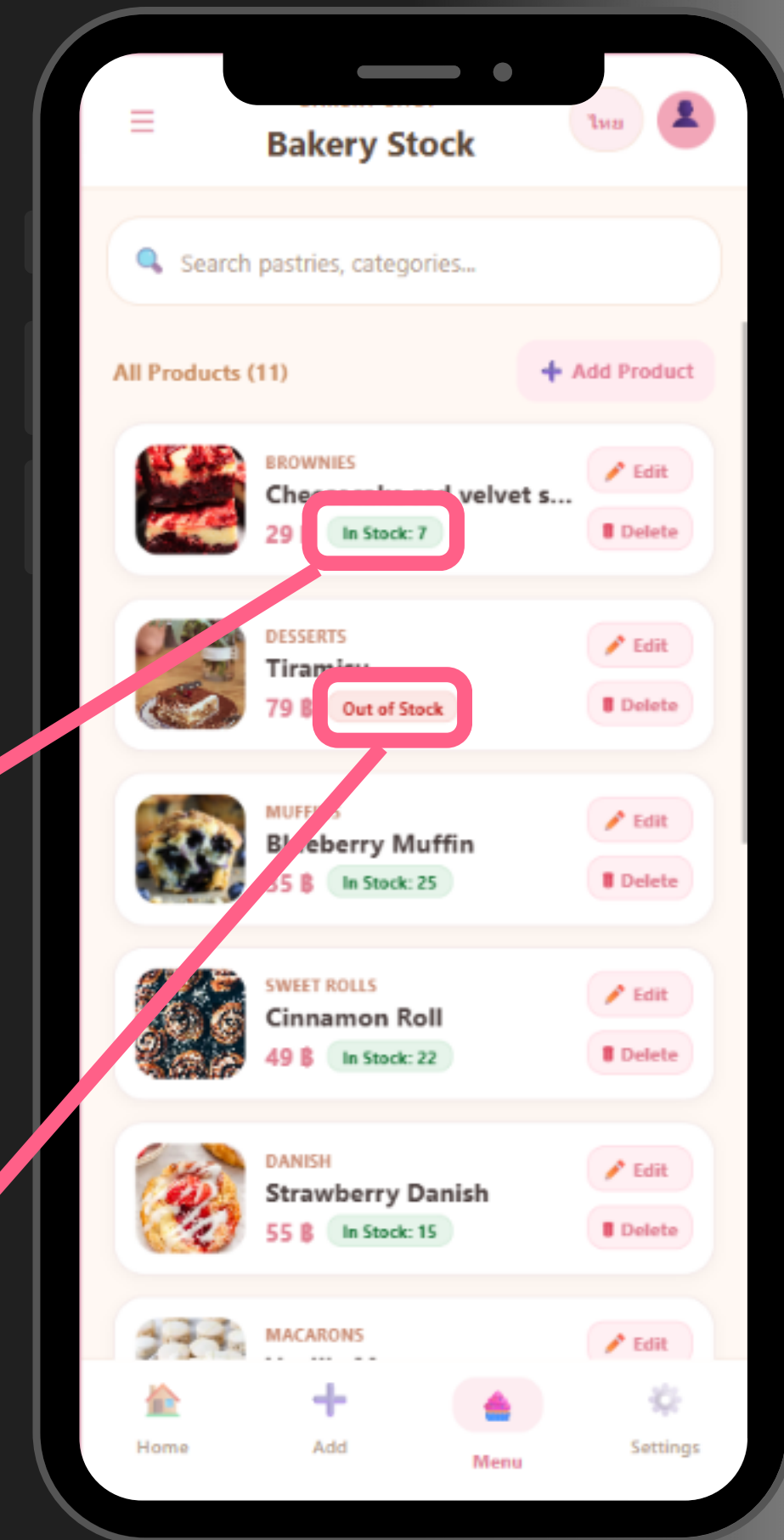
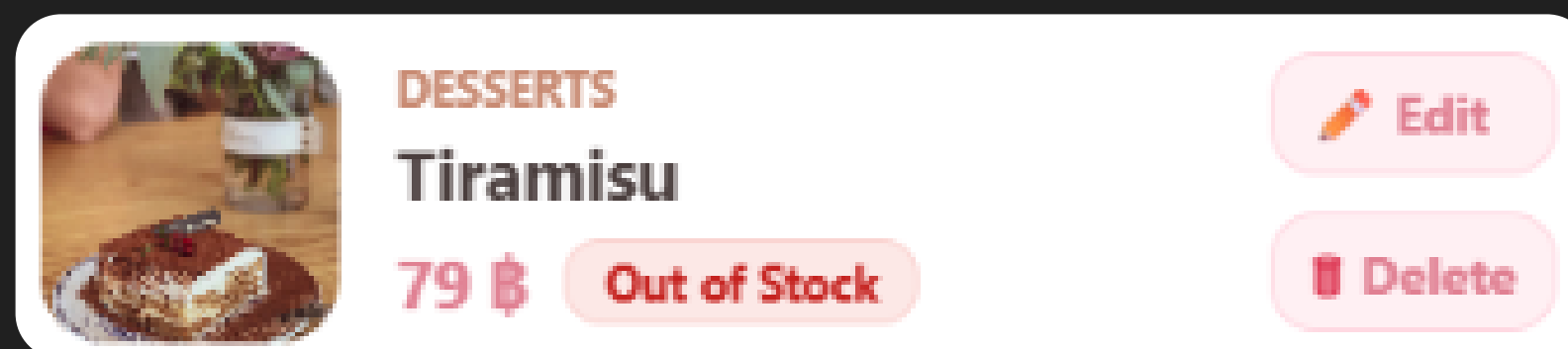
Features & Functions:

- Stock Quantity Monitoring
- Product Availability Tracking
- Real-Time Inventory Information

Indicates that the product is currently available and displays the remaining quantity in stock.



Indicates that the product is unavailable and requires restocking before it can be sold or managed.





Thank You

Inventory Management System



Wipawin Tungwattanakittikul
6730202424

https://github.com/Nwwwpw/Inventory_